



Practica Parcial 2

1er fecha 2026

Conceptos y Paradigmas de Lenguajes de Programación - 2026 - Segundo Parcial 1er Fecha. T1
12/06/2026

Realice el parcial con lapicera, de otra forma se desaprobará el/los ejercicio/s. Se considera presentismo cuando se realiza completamente un ejercicio.

(13)

Apellido: GONZALEZ

NroAlumno: 2300512

NOTA:

Aprobado

Nombre: AGUSTIN

Corrigió: Linera

Ej1 a ^b B	Ej2 a ^b B	Ej3 a ^b B	Ej4 B	Ej5 a ^b R B	Ej6 a ^b M M
----------------------	----------------------	----------------------	-------	------------------------	------------------------

Ejercicio 1 (10 ptos)

Responder V o F y justificar. Marque con un círculo la respuesta y justifique en hoja aparte.

- A. El concepto de variable en un lenguaje funcional es idéntico a los lenguajes procedurales **V(F)**
B. Si un lenguaje es compilado, implica que es fuertemente tipado. **V(F)**

Ejercicio 2 (10 ptos)

- A. Definir mutabilidad e inmutabilidad respecto a un dato.
B. Escriba ejemplos en python de ambos tipos de datos, mutables e inmutables.

Ejercicio 3 (20 ptos)

- A. Describa la semántica del tipo de variables punteros.
B. Describa ventajas y desventajas que surgen de usar variables tipo punteros

Ejercicio 4 (10 ptos)

Describan los tipos de herencia que existen y dé ejemplos de lenguajes que soporten cada una.

Ejercicio 5 (20 ptos) Sea el siguiente código escrito en Pascal like

```

1. Procedure main
2. var
3. a: array(1..10) of integer;
4. i: integer; z: integer;
5. Procedure EjecPrueba (tipo_pasaje h: integer, tipo_pasaje m: integer)
6.   Begin
7.     z:=1; m:=m+1;  $\rightarrow A(1)=5+1=6$ 
8.     h:=m+z  $\rightarrow h=6+1=7$ 
9.     z:=z+1;  $\rightarrow z=2$ 
10.    m:=m+z+a(3);  $\rightarrow m=7+2+4=13$ 
11.    z:=z*2; a(4):=a(3)-1;  $\rightarrow z=4, A(4)=7-1=6$ 
12.    write (m,a(4));
13.    m:=m+1;
14.  End;
15. Begin
16.  For i:=1 to 10 a(i):=i+1;
17.  z:=2; i:=2;
18.  EjecPrueba (z, a(z+i));
19.  For i:=1 to 10 write (a(i));
20. End.
```

- A. (10p) Indique que tipo de pasaje de parámetros se debería usar si queremos que en la impresión realizada $\rightarrow$ Explíme brevemente en la línea 12,
a.1- ambas variables impriman valores distintos.
a.2- ambas variables impriman valores iguales.
- B. (10p) Plantee diferencias, relacionada con la forma de implementación de cada uno y los resultados sobre este ejemplo considerando los siguientes tipos de pasajes parámetros: referencia y por valor. Puede realizar la pila y hacer comentarios en cada caso

Realice el parcial con lapicera, de otra forma se desaprobará el/los ejercicio/s. Se considera presentismo cuando se realiza completamente el examen.

Ejercicio 6) (30 ptos)

a) (20 p) Dado el siguiente código en python, indique cuáles son los caminos de ejecución correctos. Justifique su respuesta, indicando para la opción seleccionada cuáles son los textos que se imprimen en pantalla

```
1 def funcionGetAlturas(nombre):
2     try:
3         alturas = {'Juan': 180, 'Carlos': 160, 'Lorena': 170}
4         print(alturas[nombre])
5
6     except KeyError:
7         print("No se posee la altura del nombre ingresado")
8     else:
9         print("Imprime mensaje igual")
10        print(alturas["Osvaldo"])
11    except Exception:
12        print("Otro nombre que no existe")
13
14
15 def funcionMultiploDe2():
16     try:
17         input1 = input() //esta función lee un texto ingresado por consola
18         x = int(input1)*2
19     except TypeError:
20         print("El valor Ingresado no puede multiplicarse, ingrese un número")
21     else:
22         print("El doble del valor ingresado es "+x)
23     finally:
24         print("Esto solo se imprime si la multiplicación fue válida")
25
26
27 funcionGetAlturas("Miguel")
28 funcionMultiploDe2()
```

- A. Se invoca a funcionGetAlturas, se levanta KeyError, se imprime "No se posee la altura del nombre ingresado", luego se imprime "Imprime mensaje igual" y vuelve a levantar la excepción. Esta se propaga dinámicamente, y al no encontrar manejador el código termina en error. X
- B. Se invoca a funcionGeAlturas, se levanta KeyError, se imprime "No se posee la altura del nombre ingresado" y termina la función. Luego se invoca funcionMultiploDe2, se lee por pantalla "5", se ejecuta el else, el finally y el código termina normalmente. ✓
- C. Se invoca a funcionGetAlturas con el parámetro, se levanta KeyError, se imprime "No se posee la altura del nombre ingresado" y termina la función. Luego se invoca funcionMultiploDe2, se lee por pantalla "5", se ejecuta el finally y el código termina normalmente. X
- D. Se invoca a funcionGetAlturas, se levanta KeyError, se imprime "No se posee la altura del nombre ingresado", luego se intenta ejecutar la línea 11 y vuelve a levantar la excepción. La misma se maneja en el except de la línea 12, y la función termina. Luego se invoca funcionMultiploDe2, se lee por pantalla "5", se ejecuta el else, el finally y el código termina normalmente. X
- E. Se invoca a funcionGetAlturas, se levanta KeyError, se imprime "No se posee la altura del nombre ingresado", luego se imprime "Imprime mensaje igual" y vuelve a levantar la excepción. La misma se maneja en el except de la línea 12, y la función termina. Luego se invoca funcionMultiploDe2, se lee por pantalla "Hola", se ejecuta el else, el finally y el código termina normalmente. X
- F. Ninguna de las anteriores
- b) (10 p) Realice un pseudocódigo que implemente el código anterior en PL/1

Ejercicio 1

A) Falso, el tipo de variables en lenguajes funcionales se asocia más a una variable "matemática" y no a una celda de memoria. Las variables no pueden cambiar su valor.

B) Falso, ejemplos como C o JavaScript demuestran que un lenguaje puede ser compilado y ser de tipo dinámico. Por ejemplo en ambos hay conversión implícita.

Ejercicio 2:

A) La mutabilidad respecto a un dato se define como la capacidad de que este pueda cambiar su valor luego de ser definido, es decir, durante la ejecución. La inmutabilidad, por el contrario, una vez definido el dato este no puede cambiar.

B) Ejemplo mutable → `lista = [1, 2, 3]`
`lista.append(4)`

Ejemplo inmutable → `A = 'HOIA'`; `A += 'CHAU'`
↳ Los strings en python no se pueden cambiar.

Ejercicio 4:

Los tipos de herencia que existen son:

- Herencia simple: Una clase puede heredar comportamiento y variables de instancia de solo una clase padre como se ve en el ejemplo.

- Herencia múltiple: Una clase puede heredar comportamiento y variables de instancia de dos o más clases padre.

Lenguajes como Java soportan herencia simple y otros como python soportan tanto herencia simple como múltiple.

Ejercicio 3:

A) Las variables de tipo puntero son aquellas, que como los punteros, apuntan a una dirección de memoria. Esta dirección a la que se encuentran ligados puede contener datos primitivos como int, boolean, etc o tipos T, donde T también apunta de la misma forma y por lo tanto genera una lista enlazada.

B) X se pasa a H por referencia y a M por valor la pila quedaria con esto modificada anualmente. H comienza la referencia a Z, generando que quita a donde quita la variable pasada por parametro y M simplemente copia el valor del parametro, en el caso de pasar ambos por valor, generacion de datos parametricos como copias y no modificaciones al parametro real y si pasas ambos por referencia si generacion un enlace y cada modificacion sobre el parametro se vea reflejada en el parametro real, para que

Ejercicio 6:

A) La opcion correcta es la ~~B~~ ^D y el flujo ^{que garantiza} es el siguiente:

- Se invoca a "funcionGetAlturas" con "miguel" como parametro
- Se entra al try y luego al try-except
- Se lanza la excepcion KeyError y como tiene manejo de excepciones se genera el print "no se pudo la altura del nombre ingresado" (línea 8) y termina en el bloque try-except
- Se invoca a la funcion "funcionMultiplicar" ^{se expande continue en} ~~línea 11~~ ^{línea 11}
- Entra al try y genera a int lo ingresado por teclado
- No lanza excepcion por lo que se genera el else que imprime la linea 20, "el valor doble del valor ingresado es X" o sea 10
- Se genera el finally ya que este siempre se genera e imprime "esto solo se imprime si la multiplicacion fue valida" (línea 22)
- Se termina la ejecucion

B) PROCEDURE GETALTURAS(NOMBRE)

~~Alturas = [Juan: 180, Carlos: 160, Maria: 170]~~
~~Alturas = [Juan: 180, Carlos: 160, Maria: 170]~~

~~Alturas = [Juan: 180, Carlos: 160, Maria: 170]~~
~~Alturas = [Juan: 180, Carlos: 160, Maria: 170]~~

ALTURAS 1 = [JUAN, CARLOS, MARIA]

ALTURAS 2 = [180, 160, 170]

FOR I = 1 TO 3

IF ALTURA < 3 < 7 NOMBRE SIGNAL CONDITION KEY ERROR

ELSE

PUT SUM (ALTURAS[i]) || WRITE

ON CONDITION KEY ERROR

PUT SUM ("NO SE PUEDE ALTURA DEL NOMBRE INGRESADO")

SIGNAL CONDITION KEY ERROR

ELSE

PUT SUM ("IMPRIME MENSAJE (igual)")

END

Papel de fibra de caña de azúcar.

1er fecha 2025

Conceptos y Paradigmas de Lenguajes de Programación - Seg. Parcial 1ra Fecha. T1.- 13/06/2025
 Realice el parcial con lapicera, de otra forma se desaprobará el/los ejercicio/s.
 Se considera presentismo cuando se realiza completamente un ejercicio.

Legajo: 2300110					Comité:					
Apellido y Nombres: Guaymas Marcos Julián										
1	a	B	b	B						
2	a	/	b	B						
3	a	B	b	B						
4	a	B	b	B	c	B	d	B	e	B
Resultado Final:										Apróbatelo

Ejercicio 1 (30p)

Realice la pila de ejecución para el siguiente código:

a) por cadena estática b) por cadena dinámica **Nota:** La forma de evaluación de este lenguaje es de izquierda a derecha.

Program Main;
 Var h, m, n, o: integer;
 a, b: array[1..3] of integer;

Procedure Uno(val-res m: integer; nombre n: integer);
 var h: integer;
begin
 h:=5;
 m:= f + 2;
 n:= n + 3;
 o:=1;
 h:= n + 10;
end;

Function F: integer;
 Var n: integer;
Begin
 n:=1+h;
 if (m < 3 and h > 0) then
 begin
 h(1):= b(1) + 1;
 m:= m + 1;
 n:=2;
end;

if (n < 3) then
begin
 a(n):=a(n)+b(1)-2;
 a(n):=a(n) * 2;
 n:=1;
end
else
begin
 n:=3;
end
 return b(n);
end //de la función F

begin //del Main
 m:= 1; n:= 1; h:=1;
 for o:=1 to 3 do begin
 a(o):= o+1;
 b(o):= o * 2;
end;
 Uno(a(o), b(o));
 for o:=1 to 3 do write (a(o), b(o));
end.

Ejercicio 2

a) (10pts) Clasifique las siguientes estructuras de datos de acuerdo a lo visto en la práctica. Justifique en cada caso:

i) Python

```
class Moto:
    def __init__(self):
        self.patente = None
        self._anio = None
        self._modelo = None
        self._kms = []
    @property
    def anio(self):
        return self._anio
    @anio.setter
    def anio(self, valor):
        self._anio = valor
    def get_kms(self, km):
        return self._kms[km]
```

ii) Pascal

```
type
PVertice = ^Vertice;
PAdyacente = ^Adyacente;

// Lista de adyacencia (punteros a otros vértices)
Adyacente = record
    destino: PVertice;
    siguiente: PAdyacente;
end;

// Nodo del grafo
Vertice = record
    id: Integer;
    adyacentes: PAdyacente;
    siguiente: PVertice;
end;
```

Ejercicio 2 (10 pts) Responda si las siguientes afirmaciones son V o F. Justifique en cada caso

- No existen lenguajes compilados que sean débilmente tipados, básicamente por poder chequear estáticamente el tipo
- En java implementar una clase, asegura de por sí que ya se posee encapsulamiento y ocultamiento.

Ejercicio 3

- (15 pts) Dado el siguiente código en Java, establezca cuáles de las opciones indicadas más abajo son válidas como camino de ejecución. Justifique su selección con una breve descripción del flujo de ejecución, caso contrario no se considerará válida la respuesta)

```

1 public class ParcialExcepciones {
2     public static void main(String[] args) {
3         try{
4             for (int i = 0; i < 5; i++) {
5                 if(i==1){
6                     System.out.println(Integer.toString(i));
7                     relanzador(i);
8                 }
9                 else{
10                    if(i==2 || i==3){
11                        switch(i){
12                            case 2:
13                                System.out.println(Integer.toString(i));
14                                relanzador(i);
15                                break;
16                            case 3:
17                                System.out.println(Integer.toString(i));
18                                relanzador(i);
19                                break;
20                        }
21                    }
22                    else{
23                        System.out.println(Integer.toString(i));
24                        relanzador(i);
25                    }
26                }
27            }
28        }
29        catch(ThirdException | FourthException e){
30            System.out.println(e.getMessage());
31        }
32    }
33    static void relanzador(int i) throws ThirdException, FourthException {
34        try {
35            try {
36                switch(i){
37                    case 1:
38                        throw new FirstException(Integer.toString(i));
39                        break;
40                    case 2:
41                        throw new SecondException(Integer.toString(i));
42                        break;
43                    case 3:
44                        throw new ThirdException(Integer.toString(i));
45                        break;
46                    default:
47                        throw new FourthException(Integer.toString(i));
48                        break;
49                }
50            }
51            catch (SecondException e) {
52                ThirdException e1=new ThirdException(Integer.toString(i));
53                throw e1;
54            }
55        }
56        catch (FirstException e) {
57            FourthException e1=new FourthException(Integer.toString(i));
58            throw e1;
59        }
60    }

```

- i) Se imprime en pantalla "0", luego "0" y termina ✓
 ii) Se imprime en pantalla "0", "0", "1", "1", "2", "2", "3", "3", "4", "4" y luego termina
 iii) Se imprime en pantalla "0", "0", "1", "1", "2", "3", "3" y luego termina
 iv) Ninguna de las anteriores
 b) (15 pts) Indique si iniciando el for en 1 en vez de 0, el resultado es el mismo. Indique qué se imprime en ese caso

Ejercicio 4

(20pts). Marcar si son verdaderas o falsas las siguientes afirmaciones. Acompañar la respuesta con una justificación, caso contrario, NO se tomarán como válidas

- | | | | |
|----|---|-----|-----|
| a. | La unión y la unión discriminada son igualmente seguras | V | F ✓ |
| b. | Java tiene un equivalente a la sentencia YIELD de python | V | F ✓ |
| c. | El switch en Java no requiere la cláusula default de manera obligatoria | V ✓ | F |
| d. | En PL/1 si se genera una excepción, se ejecuta el manejador correspondiente y luego el control se pasa inmediatamente a la rutina padre de aquella donde se generó la excepción | V | F ✓ |
| e. | La sentencia finally de python en el manejo de excepciones se ejecuta siempre y cuando no se haya levantado una nueva excepción dentro de un except | V | F ✓ |

-Ejercicio 1:

Estática		Dinámica	
<pre> (*1) REG ACTIV Main h=1 m=2,2 n=1 o= 1,3,4,1..3 a(1)=2 a(2)=3,4,8 a(3)=4,5 b(1)=2,3 b(2)=4 b(3)=6,9 Procedure Uno Function F VR </pre>	<pre> m = 1, n = 1, h = 1 for o = 1..3 do begin a(o) = o + 1 b(o) = o * 2 Uno(a(o), b(o)) a(3), b(3) -> 4 ->6 for o = 1..3 do write(a(o),b(o)) Imprime (2,3) (8,4) (5,9) </pre>	<pre> (*1) REG ACTIV Main h=1 m=1 n=1 o= 1,3,1,1..3 a(1)=2 a(2)=3 a(3)=4,8 b(1)=2 b(2)=4 b(3)=6,9 Procedure Uno Function F VR </pre>	<pre> m = 1, n = 1, h = 1 for o = 1..3 do begin a(o) = o + 1 b(o) = o * 2 Uno(a(o), b(o)) a(3), b(3) -> 4 ->6 for o = 1..3 do write(a(o),b(o)) Imprime (2,2) (3,4) (8,9) </pre>
<pre> (*2) REG ACTIV Uno PR LE(*1) LD(*1) m=4,5 n=5,b(o) h=5,13 VR 3 </pre>	<pre> h = 5 m = f + 2 = 3 + 2 = 5 n = n + 3 = b(3) + 3 = 9 o = 1 h = n + 10 = b(1) + 10 = 3 + 10 = 13 </pre>	<pre> (*2) REG ACTIV Uno PR LE(*1) LD(*1) m=4,8 n=5,b(o) h=5,12 VR 6 </pre>	<pre> h = 5 m = f + 2 = 6 + 2 = 8 n = n + 3 = b(3) + 3 = 9 o = 1 h = n + 10 = b(1) + 10 = 2 + 10 = 12 </pre>
<pre> (*3) REG ACTIV F PR LE(*1) LD(*2) n=2,2,1 VR </pre>	<pre> n = 1 + h = 1 + 1 = 2 b(1) = b(1) + 1 = 3 m = m + 1 = 1 + 1 = 2 n = 2 a(n) = a(2) = a(2) + b(1) - 2 = 4 a(2) = a(2) * 2 = 8 n = 1 return b(n) -> b(1) = 3 </pre>	<pre> (*3) REG ACTIV F PR LE(*1) LD(*2) n=2,3 VR </pre>	<pre> n = 1 + h = 1 + 5 = 6 n = 3 return b(n) -> b(3) </pre>

-Ejercicio 2:

a)

i) Python

class Moto → Producto cartesiano, el constructor equivale a un registro que agrupa campos de distintos tipos, Correspondencia finita en el atributo _kms ya que es una lista que mapean indices enteros a valores almacenados

ii) Pascal

PVertice → Producto cartesiano por parte de Adyacente y Vertice, ya que es un registro que agrupa campos de distintos tipos, Recursion por parte de PVertice y Padyacente ya que son listas enlazadas que contienen referencias a otras instancias del mismo tipo

b)

i) No existen lenguajes compilados que sean debilmente tipados, basicamente por poder chequear estaticamente el tipo

-Falso, que un lenguaje sea compilado no significa que automaticamente lo vuelva debilmente tipado. Existen lenguajes compilados debilmente tipados como C o JS que hacen conversiones implicitas, donde en C puedes sumar o restar un valor a una variable de tipo puntero o en JS si sumas "10" + 5 te puede dar "105"

ii) En java implementar una clase asegura de por si que ya se posee encapsulamiento y ocultamiento

-Falso, cuando creas una clase en java puedes definir las variables de instancia como publicas y eso rompería con la clausula de encapsulamiento y ocultamiento, ya que cualquier otra clase podría acceder a esa variable de instancia

-Ejercicio 3:

a) La opción correcta es la i) , El flujo de ejecución es el siguiente:

-Se entra al try-catch de la línea 4, donde internamente hay un for de 0 a 5 que se recorre por la variable i inicializada en 0

-Al ser i = 0, se entra por el último else de la línea 23 y se imprime (0) y se llama al método relanzador con 0 como parámetro

-El método relanzador puede lanzar las excepciones Third y Fourth ya que lo declara en el encabezado con throws

-Este método tiene 2 try-catch anidados, uno que arranca en la línea 35 y el otro en la 36, al evaluar todos los posibles caminos a partir de la línea 37 se entra por el default ya que i = 0 y todos los otros casos darían falso

-Se lanza la excepción de tipo FourthExcepcion con mensaje "0"

-Se busca un manejador y no se encuentra dentro del try-catch interno, por lo que se propaga al try-catch externo y tampoco se encuentra. Se termina propagando al main donde hay un manejador en la línea 29 y se imprime el mensaje "0"

-Se finaliza la ejecución del programa

b) Si se inicializa el for en 1 en vez del 0 el resultado no sería el mismo ya que entraría al primer if y se imprimiría "1" y se llamaría a relanzador con 1 como parámetro, donde en este mismo se entraría al primer try-catch, luego al try-catch anidado y entraría el primer case, donde lanzaría la excepción FirstException (línea 39), se buscaría el manejador que se encuentra en el try-catch externo, se lanza la excepción FourthException que se encuentra en el main (línea 29) y se imprimiría nuevamente "1" y termina

-Ejercicio 4:

a) La unión y la unión discriminada son igualmente seguras

-Falso, la union discriminada es mas segura ya que agrega un descriptor enumerativo que te dice que campo tiene valor, evitando problemas de acceder a campos nulos

b) Java tiene un equivalente a la sentencia YIELD de python

-Falso, en java no existe esto y solo existe el return que devuelve un valor unico y listo, en cambio el yield devuelve una secuencia de valores y ademas guarda el estado interno

c) El switch en Java no requiere la capsual default de manera obligatoria

-Verdadero, esta sentencia es opcional y su funcion es atrapar aquellas ocurrencias donde ningun valor coincidio con alguno de la lista previamente definido

d) En PL/1 si se genera una excepcion, se ejecuta el manejador correspondiente y luego el control pasa inmediatamente a la rutina padre de aquella donde se genero la exepcion

-Falso, luego de ejecutar el manejador correspondiente se continua desde la linea siguiente donde se genero la excepcion, ya que PL/1 utiliza el modelo de reasuncion

e) La sentencia finally de python en el manejo de excepciones se ejecuta siempre y cuando no se haya levantado una nueva excepcion dentro de un except

-Falso, la sntencia finally se ejecuta siempre, sin importar si ocurrio una excepcion o no. Lo que se menciona es el comportamiento de la sentencia else

1er fecha 2023

Realice el parcial con lapicera, de otra forma se desaprobará el/los ejercicio/s.
Se considera presentismo cuando se realiza completamente un ejercicio.

```

    public float getPromedio(){
        return this.promedio;
    }
}

```

ii) C

```

typedef struct _nodoArbol {
    void *info;
    struct _nodoArbol *hijoIzq;
    struct _nodoArbol *hijoDer;
} nodoArbol;

typedef struct _arbolBinario {
    int valor_guardado;
    nodoArbol *raiz
} arbolBinario;

```

b) (10 pts) Responda si las siguientes afirmaciones son V o F. Justifique en cada caso

- i) Los lenguajes con sistema de tipos fuerte son siempre compilados **F**
- ii) Las tuplas de python son un ejemplo de producto cartesiano **F**
- iii) La unión y la unión discriminada no son seguras en ejecución **V**

Ejercicio 3

a) (15 pts) Dado el siguiente código en Java, establezca cuáles de las opciones indicadas más abajo son válidas como camino de ejecución. Justifique con una breve descripción del flujo de ejecución, caso contrario no se considerará válida la respuesta)

```

1 public class Java7MultiplesExceptions {
2
3     public static void main(String[] args) {
4         try{
5             for (int i = 1; i < 4; i++) {
6                 if(i==1){
7                     System.out.println(Integer.toString(i));
8                     rethrow("Primera");
9                 }
10                else{
11                    if(i==2){
12                        System.out.println(Integer.toString(i));
13                        rethrow("Segunda");
14                    }
15                    else{
16                        if(i==3){
17                            System.out.println(Integer.toString(i));
18                            rethrow("Tercera");
19                        }
20                    }
21                }
22            }
23        }catch(ThirdException e){
24            System.out.println(e.getMessage());
25        }
26    }
27
28    static void rethrow(String s) throws ThirdException {
29        try {
30            try {
31                if (s.equals("Primera")){
32                    throw new FirstException("Primera excepción");
33                }
34            }
35            else{
36                if (s.equals("Segunda")){
37                    throw new SecondException("Segunda excepción");
38                }
39            }
40            else{
41                throw new ThirdException("Tercera excepción");
42            }
43        }catch (SecondException e) {
44            ThirdException e1=new ThirdException("Tercera excepción");
45        }
46    }
47 }

```

Escaneado con CamScanner

Conceptos y Paradigmas de Lenguajes de Programación - 2023 - Segundo Parcial 1ra Fecha. T1
16/06/2023

Realice el parcial con lapicera, de otra forma se desaprobará el/los ejercicio/s.
Se considera presentismo cuando se realiza completamente un ejercicio.

```

44         throw e1;
45     }
46 }catch (FirstException e) {
47     ThirdException e1=new ThirdException("Tercera excepción");
48     throw e1;
49 }
50 }
51 }
    
```

- i) Se imprime en pantalla "1" y luego "tercera excepción" y luego termina
 ii) Se imprime en pantalla "1", "Primera excepción", "2", "segunda excepción", "3", "tercera excepción" y luego termina
 iii) Se imprime en pantalla "1", "Tercera excepción", "2", "Tercera excepción", "3", "tercera excepción" y luego termina
 iv) Ninguna de las anteriores

- b) (10 pts) Indique si el resultado de intercambiar las líneas 4 y 5 (es decir, el try fuera del for) genera el mismo resultado de impresión. Justifique

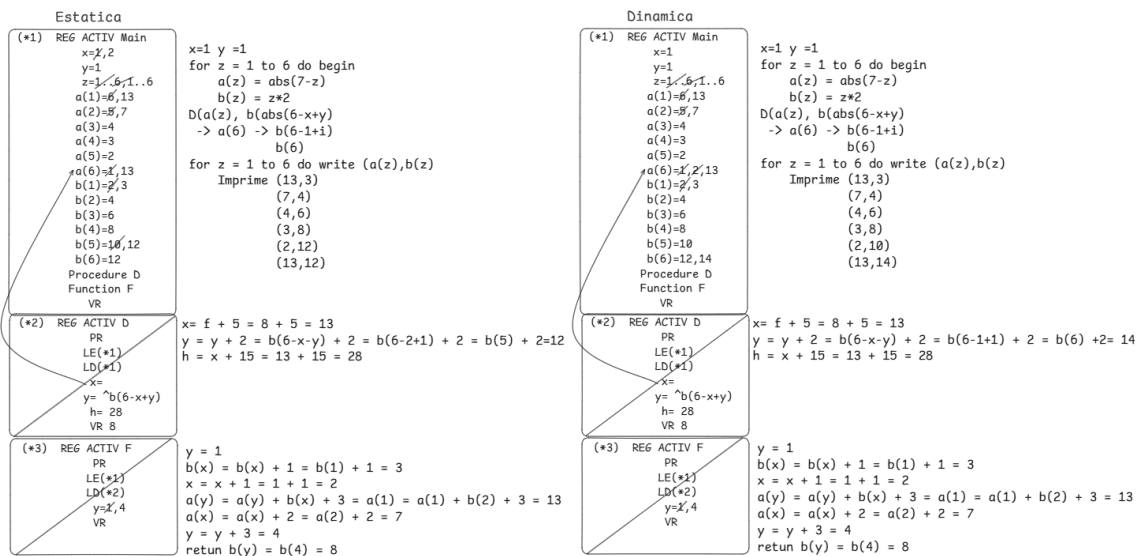
Ejercicio 4

(25pts). Marcar si son verdaderas o falsas las siguientes afirmaciones. Acompañar la respuesta con una justificación, caso contrario, NO se tomarán como válidas

- a. La sentencia for de ADA y Pascal son igualmente seguras V F
 b. El if de circuito corto puede prevenir errores en ejecución V F
 c. La sentencia yield de python equivale a hacer return V F
 d. En PL/1 si se genera una excepción, se ejecuta el manejador correspondiente y el control es pasado inmediatamente al programa principal V F
 e. La sentencia else de python en el manejo de excepciones se ejecuta solo si no se encontró ningún manejador asociado a la excepción en cuestión V F

Escaneado con CamScanner

Ejercicio 1:



Ejercicio 2:

a)

i) Java

class Alumno → Producto cartesiano ya que agrupa campos de diferentes tipos en la clase Alumno, en este caso serian el nombre, apellido, edad, promedio y domicilio

ii) C

struct → Producto cartesiano, agrupa campos de diferentes tipos, en este caso serian info y da la particularidad que los otros dos campos son de tipo puntero por lo que tambien hay Recursion, donde un dato de tipo T puede contener datos de tipo T, generando por ejemplo una lista enlazada creciente de forma arbitraria

b)

i) Los lenguajes con sistemas de tipos fuerte siempre son compilados

-Falso, esto no es verdad ya que existe lenguajes de tipos fuerte que son dinamicos, como por ejemplo Python

ii) Las tuplas de python son un ejemplo de producto cartesiano

-Falso, las tuplas de python son un ejemplo de correspondencia finita, ya que es un conjunto de valores de un dominio DT que al accederlo por un indice devuelve un valor de un tipo de dominio RT

iii) La union y la union discriminada no son seguras en ejecucion

-Verdadero, son inseguras ya que puede ocurrir que se intente acceder a un valor que no se debe acceder, provocando un error. En union discriminada esto se resolveria usando el descriptor enumerativo y accediendo solo a los campos que se debe o a campos con valor, pero como no es obligatorio usarlo por default ambas uniones son inseguras

Ejercicio 3:

a)

-La opcion correcta es la i) y el flujo es el siguiente:

-Se inicializa i=1 en el for de 1 a 4 dentro del try del main (linea 4)

-Como i=1 entra en el primer if, se imprime "1" y se llama a rethrow con "Primera" como parametro

-Rethrow es un metodo que puede lanzar o propagar la excepcion

ThirdExcepcion pero cuando se entra al primer try, y luego al try-catch anidado coincide con el primer if

- Se lanza una excepcion de tipo FirstExcepcion y se busca un manejador estaticamente que no va a ser encontrado
- Se propaga al try-catch externo y se encuentra un manejador de esta excepcion (linea 46) y se ejecuta ThirdException con "Tercera excepcion" como parametro
- Se propaga dinamicamente y se encuentra un manejador (linea 23), donde se imprime el mensaje "Tercera excepcion"
- Se finaliza la ejecucion del programa y el for no continua ya que se genero una excepcion y el bloque donde fue generada la misma es terminado

b) El resultado no es el mismo ya que si el for esta afuera va a realizar todas las iteraciones. En cambio, si lo dejamos como esta, el programa termina una vez se genera la excepcion y el for no se seguiria ejecutando

Ejercicio 4:

- a) La sentencia for de ADA y Pascal son igualmente seguras
 - Falso, esto es falso ya que en Pascal se permite modificar la variable de control y podria generar algun error, esto en ADA no ocurre
- b) El if de circuito corto puede prevenir errores en ejecucion
 - Verdadero, ya que al confirmar que la primera condicion es falsa ya corta con la ejecucion. Si por ejemplo, en la segunda condicion se accedia a una posicion de vector invalida y esto ya era descartado por la primer opcion, nos salvaria de un error de acceso a una posicion erronea
- c) La sentencia yield de python equivale a hacer un return
 - Falso, la sentencia yield devuelve una secuencia de valores y ademas mantiene un estado en diferentes llamados, es diferente al return que solo devuelve 1 valor y no mantiene estado
- d) En PL/1 si se genera una excepcion, se ejecuta el manejador correspondiente y el control es pasado inmediatamente al programa principal
 - Falso, como PL/1 usa el metodo de reasuncion, luego de ejecutar el manejador correspondiente se sigue en la siguiente linea de codigo a donde ocurrio la excepcion
- e) La sentencia else de python en el manejo de excepciones se ejecuta solo si no se encontro ningun manejador asociado a la excepcion en cuestion

-Falso, la sentencia else de python se ejecuta solo si no se genero una excepcion. El default es lo que se ejecuta si no se encontro previamente un manejador asociado